

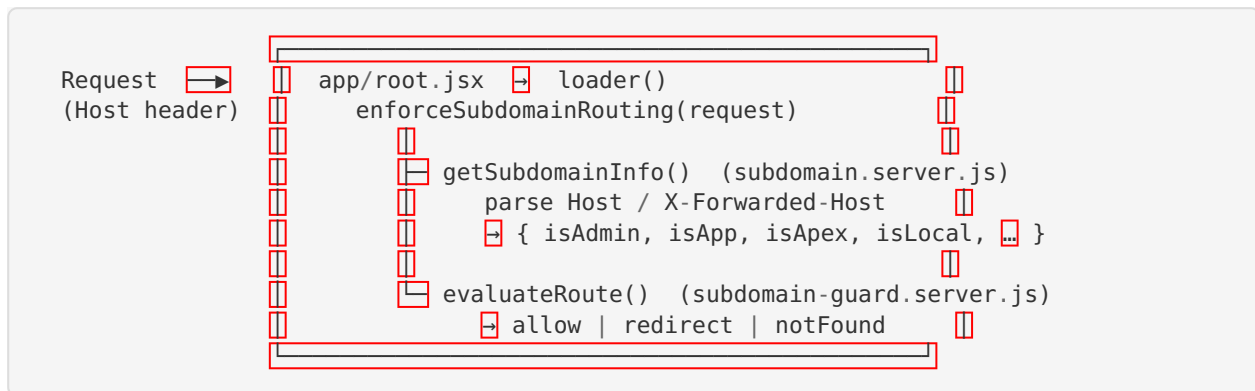
Subdomain Deployment Guide

How to deploy the **Luciteria Collector Cabinet** as two logically separate sites from a single codebase, split by subdomain:

Subdomain	Audience	Allowed routes
app.luciteria.com	Consumers	/, /app/*, /onboarding/*, /wishlist/*, ...
admin.luciteria.com	Internal operators	/admin/* only (root redirects to /admin/login)
luciteria.com / www.*	Apex / marketing	No restriction (serves the same app)

The split is enforced **in-app** by a server-side guard, so it works whether you run one deployment that answers both hostnames, or two independent deployments.

1. How it works (architecture)



Two files implement the logic:

- **app/lib/subdomain.server.js** — pure host parsing. Reads `Host` (or the proxy `X-Forwarded-Host`) header, strips ports, and classifies the request as `admin`, `app`, `apex/www`, `local`, or `unknown` host.
- **app/lib/subdomain-guard.server.js** — the routing rules. A pure `evaluateRoute()` decides `allow` / `redirect` / `notFound`, and `enforceSubdomainRouting()` throws the matching `Response`.

It is wired into **app/root.jsx** (runs on every request) and also into **app/routes/admin.jsx** (the `/admin/*` layout) so that on the consumer host a clean **404** wins over the admin auth redirect.

The rules

Host	Path	Result
admin.*	/	302 → /admin/login
admin.*	any non- /admin	302 → /admin/login
admin.*	/admin/*	allow (auth gate still applies)
app.*	/admin or /admin/*	404 Not Found
app.*	everything else	allow
apex / www.*	anything	allow (no enforcement)
localhost / IP / *.local / pre-view hosts	anything	allow (no enforcement)

Note: /app/admin/* (the Shopify-embedded admin) is **not** affected — the guard only matches the standalone /admin portal, never /app/...

2. Environment variables

All are **optional** — the defaults match luciteria.com.

Variable	Default	Purpose
<code>ROOT_DOMAIN</code>	<code>luciteria.com</code>	Registrable domain. Set to your real domain (e.g. <code>luciteria.com</code>).
<code>ADMIN_SUBDOMAIN</code>	<code>admin</code>	Label that means “admin host”.
<code>APP_SUBDOMAIN</code>	<code>app</code>	Label that means “consumer host”.
<code>FORCE_SUBDOMAIN</code>	(unset)	Override detection for local testing, e.g. <code>FORCE_SUBDOMAIN=admin</code> .
<code>NODE_ENV</code>	—	<code>production</code> enables secure cookies; routing works regardless.
<code>ADMIN_SESSION_SECRET</code>	dev fallback	Set a strong secret in production (admin auth cookie).
<code>SESSION_SECRET</code>	dev fallback	Set a strong secret in production (consumer auth cookie).

Example `.env.production` :

```

ROOT_DOMAIN="luciteria.com"
ADMIN_SUBDOMAIN="admin"
APP_SUBDOMAIN="app"
NODE_ENV="production"
DATABASE_URL="postgresql://user:pass@host:5432/luciteria"
ADMIN_SESSION_SECRET("<openssl rand -hex 32>"
SESSION_SECRET("<openssl rand -hex 32>"
APP_URL="https://app.luciteria.com"

```

Because detection is host-based, **the same build and the same env file** can serve both subdomains. You do **not** need separate `ADMIN_SUBDOMAIN` per deployment.

3. DNS configuration

Point both subdomains at your host. Use the record type your host requires.

Option A — both subdomains → one deployment (simplest)

Type	Name	Value	TTL
A	app	<server-ip>	3600
A	admin	<server-ip>	3600
# (optional apex + www)			
A	@	<server-ip>	3600
CNAME	www	luciteria.com.	3600

If your platform gives you a hostname instead of an IP (Vercel, Fly, Render, Railway, Heroku, etc.) use CNAMEs:

Type	Name	Value	TTL
CNAME	app	your-app.onrender.com.	3600
CNAME	admin	your-app.onrender.com.	3600

Option B — two independent deployments

Run two instances of the same build (e.g. one container per subdomain) and point each subdomain at its own instance:

CNAME	app	consumer-instance.host.com.	3600
CNAME	admin	admin-instance.host.com.	3600

The in-app guard still applies on each instance, so even if someone hits the “wrong” instance the rules hold.

TLS: issue certificates that cover both subdomains. A wildcard

`*.luciteria.com` cert, or individual certs for `app.` and `admin.`, both work.

With Let’s Encrypt + Caddy/Traefik this is automatic.

4. Reverse proxy / forwarded headers

If a proxy (Nginx, Cloudflare, a load balancer) sits in front of the app, make sure the original host is forwarded. The guard reads `X-Forwarded-Host` first, then falls back to `Host`.

Nginx:

```
location / {
    proxy_pass http://127.0.0.1:3000;
    proxy_set_header Host $host;
    proxy_set_header X-Forwarded-Host $host;
    proxy_set_header X-Forwarded-Proto $scheme;
}
```

Caddy:

```
app.luciteria.com, admin.luciteria.com {
  reverse_proxy 127.0.0.1:3000
}
```

Caddy forwards `Host` automatically.

5. Deployment workflow

```
# 1. Install + generate Prisma client
npm ci
npx prisma generate

# 2. Apply DB migrations (production DB)
npx prisma migrate deploy

# 3. Seed admin user (first deploy only)
node prisma/seed-admin.js          # creates admin@luciteria.com

# 4. Build
npm run build                      # → remix vite:build

# 5. Start (serves both subdomains)
NODE_ENV=production \
ROOT_DOMAIN=luciteria.com \
npm run start:production          # → remix-serve ./build/server/index.js
```

The server listens on `PORT` (default `3000`). Put your proxy/TLS in front and route both subdomains to it.

Dev server caveat: `npm run dev` (Vite) blocks unknown `Host` headers with a `403 "This host is not allowed"`. That is Vite's protection, not this guard. To test real hostnames locally use the production build (`npm run build && npm run start:production`) or add the hosts to `server.allowedHosts` in `vite.config.js`. See Testing below.

6. Testing

a) Unit / logic tests (no network)

```
npx vite-node test-subdomain.mjs
# →  ALL TESTS PASSED – 61 assertions
```

Covers `admin/app/apex/www/localhost/IP/IPv6/preview/empty` hosts, deep subdomains, custom root domains, `/admin` vs `/app/admin`, asset bypass paths, redirect-loop avoidance, and `FORCE_SUBDOMAIN`.

b) Local end-to-end with the production build

```
npm run build
PORT=3100 NODE_ENV=production npx remix-serve ./build/server/index.js &

# Admin host
curl -sI -H 'Host: admin.luciteria.com' http://localhost:3100/ # 302 → /admin/login
curl -sI -H 'Host: admin.luciteria.com' http://localhost:3100/admin/login # 200

# Consumer host — admin is hidden
curl -sI -H 'Host: app.luciteria.com' http://localhost:3100/ # 200
curl -sI -H 'Host: app.luciteria.com' http://localhost:3100/admin/users # 404
```

c) Force a subdomain in the Vite dev server

Add the hosts to `vite.config.js` so the dev server stops returning 403:

```
// vite.config.js
export default defineConfig({
  server: {
    allowedHosts: ["admin.luciteria.com", "app.luciteria.com"],
  },
  // ...
});
```

Then add to `/etc/hosts` :

```
127.0.0.1 app.luciteria.com admin.luciteria.com
```

...and browse to `http://admin.luciteria.com:3000` . Or skip hostnames entirely and use the env override:

```
FORCE_SUBDOMAIN=admin npm run dev # every request behaves as the admin host
FORCE_SUBDOMAIN=app npm run dev # every request behaves as the consumer host
```

7. Troubleshooting

Symptom	Likely cause / fix
403 "This host is not allowed" in dev	Vite dev-server host check. Add hosts to <code>server.allowedHosts</code> , or test with the production build. Not a guard issue.
Subdomain rules don't apply (everything allowed)	Host isn't recognised as under <code>ROOT_DOMAIN</code> . Check <code>ROOT_DOMAIN</code> , and that the proxy forwards <code>X-Forwarded-Host</code> / <code>Host</code> . Preview/staging hosts on a different domain are intentionally unrestricted.
<code>admin.*</code> keeps redirecting in a loop	Ensure <code>/admin/login</code> is reachable and not itself redirected. The guard already avoids self-redirects; confirm no extra proxy rewrite.
<code>/admin/*</code> returns 302 instead of 404 on <code>app.*</code>	<code>app/routes/admin.jsx</code> must call <code>enforceSubdomainRouting()</code> before <code>requireAdmin()</code> . Confirm the import is present.
Static assets / <code>/build/*</code> get redirected	Add the path prefix to <code>ALWAYS_ALLOW_PREFIXES</code> in <code>subdomain-guard.server.js</code> .
Works locally, fails behind Cloudflare	Cloudflare may strip/rename headers. Confirm <code>Host</code> reaches origin, or rely on <code>X-Forwarded-Host</code> .
Custom domain (not luciteria.com)	Set <code>ROOT_DOMAIN</code> (and <code>ADMIN_SUBDOMAIN</code> / <code>APP_SUBDOMAIN</code> if different) in the environment.
Secure cookies not set / login not persisting	Set <code>NODE_ENV=production</code> and strong <code>ADMIN_SESSION_SECRET</code> / <code>SESSION_SECRET</code> .

8. File reference

File	Responsibility
<code>app/lib/subdomain.server.js</code>	Parse Host header → subdomain classification (pure).
<code>app/lib/subdomain-guard.server.js</code>	Routing rules + <code>enforceSubdomainRouting()</code> middleware.
<code>app/root.jsx</code>	Calls the guard in its <code>loader</code> (every request).
<code>app/routes/admin.jsx</code>	Calls the guard before <code>requireAdmin</code> for clean 404s.
<code>test-subdomain.mjs</code>	61-assertion edge-case test suite.